

The Comprehensive Roadmap Guide

Project Management for AI-Assisted Engineering

DOT E 3010

Abstract

Creating software is not about typing code; it is about managing complexity. For business students using AI (Vibe Coding), the challenge is not syntax, but sequencing. This document breaks down the software development lifecycle into atomic, manageable steps that you can feed to an AI agent one by one. It covers the "Why" and "How" of every phase from empty folder to deployed application.

Contents

The Philosophy: You are the Project Manager	2
Why do we need a Roadmap?	2
The Mental Model: Software as a House	2
Phase 0: The Environment (The Setup)	3
0.1 The Terminal (The Command Center)	3
0.2 Virtual Environment (The Sandbox)	3
0.3 Dependencies (The Toolbelt)	3
Phase 1: The Brain (Core Logic)	3
Why start here?	3
Phase 2: The Memory (Database)	3
Schema Definition	3
The Connection	4
Phase 3: The Gateway (API)	4
The Waiter Analogy	4
Roadmap Steps	4
Phase 4: The Face (Frontend)	4
Roadmap Steps	4
Phase 5: The Deployment (Shipping)	5
Docker (The Shipping Container)	5
The Daily Workflow: How to Execute	5
Troubleshooting Encyclopedia	5
Scenario A: The "Hallucination Loop"	5
Scenario B: "It works on my machine"	5
Scenario C: The "Frankenstein" Code	5

The Philosophy: You are the Project Manager

When you use tools like Cursor, Windsurf, or ChatGPT to code, you are not the programmer. **The AI is the programmer.** You are the Project Manager (PM).

Why do we need a Roadmap?

AI has a short attention span (Context Window). If you ask it to "Build the whole app," it will try to do everything at once, run out of memory, hallucinate variables, and produce broken code.

The 10-Minute Rule

Every step in your roadmap should be small enough for the AI to complete in 10 minutes. If a step says "Build the Backend," it is too big. It must be broken down into "Setup FastAPI," "Create Database," "Write Login API."

The Mental Model: Software as a House

To understand the roadmap, visualize building a house:

- **Phase 0 (Foundation):** Clearing the land, buying tools (Setup).
- **Phase 1 (Internal Plumbing):** Pipes and wires that no one sees (Core Logic).
- **Phase 2 (Closets/Storage):** Places to put things (Database).
- **Phase 3 (Doors/Windows):** Connection points (API).
- **Phase 4 (Paint/Decoration):** What the user sees (Frontend).

Most beginners try to paint the walls (Phase 4) before laying the foundation (Phase 0). This roadmap forces you to build in the correct order.

Phase 0: The Environment (The Setup)

Goal: Prepare the digital workspace.

0.1 The Terminal (The Command Center)

The black screen with white text is not scary; it is just a direct line to your computer's operating system.

- **Why:** AI agents operate best via terminal commands.
- **Roadmap Step:** "Open Terminal and navigate to Desktop."

0.2 Virtual Environment (The Sandbox)

Concept: Python is messy. If you install a library globally, it might break other projects. A "Virtual Environment" (`venv`) is a sealed sandbox. Anything you install inside stays inside.

- **Roadmap Step:** "Initialize `venv`."
- **Roadmap Step:** "Activate `venv`."

0.3 Dependencies (The Toolbelt)

Your AI carpenter needs a hammer and saw. In Python, these are libraries like `FastAPI` or `Pandas`. We list them in a file called `requirements.txt`.

- **Roadmap Step:** "Create `requirements.txt` with necessary libraries."
- **Roadmap Step:** "Run `pip install`."

Phase 1: The Brain (Core Logic)

Goal: Pure Logic. No Web, No User, just Input -> Output.

Why start here?

If you are building a calculator app, you must first write the code that adds $1 + 1$. If that doesn't work, a beautiful interface is useless.

- **Roadmap Step:** "Write a standalone Python script `calc.py` that adds two numbers."
- **Roadmap Step:** "Run the script in terminal to verify it works."

For AI Projects: This is where you test your Prompt Engineering. Write a script that sends a prompt to OpenAI and prints the response. Verify the AI isn't talking nonsense before you build a website around it.

Phase 2: The Memory (Database)

Goal: Persistence. Making data survive a restart.

Schema Definition

You must define what the data looks like. This is called a "Model" in coding terms.

- **Roadmap Step:** "Create `models.py`. Define User class with email and password fields."

The Connection

The code needs to open a file (SQLite) or connect to a server (Postgres).

- **Roadmap Step:** "Setup `database.py` with SQLAlchemy engine."
- **Roadmap Step:** "Create a script `init_db.py` to create the tables."

Phase 3: The Gateway (API)

Goal: Exposing the Brain to the World.

The Waiter Analogy

Imagine a restaurant.

- The **Frontend** is the Customer (Menu).
- The **Backend** is the Kitchen (Cooks).
- The **API** is the Waiter.

The API takes an order (Request), brings it to the kitchen, and brings back the food (Response).

Roadmap Steps

- **Step:** "Create `main.py` with FastAPI."
- **Step:** "Create a POST endpoint `/login` that accepts email/password."
- **Step:** "Create a GET endpoint `/products` that returns a list of items from the Database."
- **Verification:** Use the "Swagger UI" (auto-generated documentation at `/docs`) to test the waiter before building the frontend.

Phase 4: The Face (Frontend)

Goal: User Interaction.

For Vibe Coding, we prefer **No-Build** Frontends.

- **Standard Way (Hard):** React, Vue, Node.js, NPM, Webpack, Transpiling. (Too many places for AI to get stuck).
- **Vibe Way (Easy):** One `index.html` file. Import styles via CDN. Write JavaScript directly in the file.

Roadmap Steps

- **Step:** "Create `static/index.html`."
- **Step:** "Add a form for Login."
- **Step:** "Write JavaScript `fetch()` function to call the `/login` API."
- **Step:** "Handle the response (e.g., Show 'Success' alert)."

Phase 5: The Deployment (Shipping)

Goal: Running it on another computer.

Docker (The Shipping Container)

Your code works on your laptop because you have specific files and settings. It won't work on your teacher's laptop. Docker solves this by packaging your OS, Python, and Code into a "Container."

- **Roadmap Step:** "Write Dockerfile." (Instructions to build the container).
- **Roadmap Step:** "Write docker-compose.yml." (Instructions to run the container + database).
- **Verification:** "Run docker-compose up and test in browser."

The Daily Workflow: How to Execute

Do not just give the AI the roadmap. Follow this loop:

1. **Context Loading:** Start a new chat. Paste the Project Summary and the *Current Roadmap* status.
2. **The Ask:** Copy **ONE** step from the roadmap. "AI, please implement Step 2.3."
3. **The Review:** Look at the code. Does it match the PRD?
4. **The Verification:** Ask the AI: "Write me a test script to prove this works." Run it.
5. **The Commit:** If it works, save the file. Check the box. Move to the next step.

Troubleshooting Encyclopedia

Scenario A: The "Hallucination Loop"

Symptom: AI writes code, you run it, it errors. AI "fixes" it, you run it, new error. AI "fixes" it, original error returns.

Solution: STOP. The AI is guessing.

1. Revert the file to the last working state.
2. Copy the *exact* error message.
3. Say: "Stop coding. Explain WHY this error is happening. Analyze the root cause."
4. Only then ask for a fix.

Scenario B: "It works on my machine"

Symptom: You send the code to a friend, it crashes.

Solution: You likely forgot to include a library in `requirements.txt` or an Environment Variable in `.env`. Always check these two files first.

Scenario C: The "Frankenstein" Code

Symptom: The file is 500 lines long and does everything (Database, UI, Logic) in one place.

Solution: Tell the AI: "Refactor this. Move the Database logic to `database.py` and the Core Logic to `core.py`." Keep files small.